MICROSERVICES COURSE

Session 3

API Gateway Advanced

JWT Authentication · Rate Limiting · Redis · Security Boundary

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

Wednesday

3:00 PM – 5:30 PM

Online Session

2.5 Hours

Phase 1

Session 3 of 29

WHERE WE ARE

Phase 1 — Foundation & Core Patterns

S1	S2	S3	S4	S5	S6	S7	S8
Architecture	Gateway Core	Gateway Adv.	Resilience CB	Resilience BH	OpenFeign	Saga+Kafka	Caching+Clinic
		● LIVE					

You are here — Session 3 of 8 in Phase 1. The Gateway exists. Today we make it smart: security and protection.

QUICK RECAP — SESSION 2

The Gateway Routes. But It Doesn't Protect — Yet.

What We Have

→ Single entry point — API Gateway :8080

→ Routes /api/products/** to Product Service

→ Global Logging Filter on every request

→ Load balancing across instances

What's Missing

→ Anyone can call any endpoint — no authentication

→ No protection against abuse — 10,000 req/sec breaks Product Service

→ No way to know WHO is calling

Today: Making the Gateway smart — adding a Security & Protection layer with JWT validation and Rate Limiting.

GlobalFilter vs GatewayFilterFactory

GatewayFilterFactory

- Creates configurable filters for routes
- Used in application.yml: "- name: ..."
- Example: RequestRateLimiter, StripPrefix
- Parameterized — same filter, different args per route

GlobalFilter (today's focus)

- Implemented as @Component, applies to ALL routes
- No configuration needed — always active
- Example: LoggingFilter (S2), JwtAuthFilter (today)
- Best for cross-cutting concerns: auth, logging

MECHANICS

Filter Execution Order — Why It Matters

1

LoggingFilter

`order = HIGHEST_PRECEDENCE`

Logs every request first — even rejected ones

→

2

JwtAuthFilter

`order = HIGHEST_PRECEDENCE + 1`

Validates token — runs after logging, before routing

→

3

Routing

`Predicate match`

Request forwarded to the matched service

→

4

RequestRateLimiter

`Filter Factory on route`

Applied per-route, checked alongside routing

CORE CONCEPT

JWT — Header . Payload . Signature

HEADER

```
{"alg":"HS256","typ":"JWT"}
```

Algorithm + token type

PAYLOAD

```
{"sub":"user123","role":"ADMIN"}
```

Claims — user data, expiry, role

SIGNATURE

```
HMACSHA256(header+payload,
secret)
```

Proves the token wasn't tampered with

The Gateway can verify the signature LOCALLY using the shared secret — no network call to an auth server needed for every request.

WHAT THE FILTER DOES

JWT Validation Flow — 7 Steps

1. Check if route is whitelisted (public routes skip validation)
2. Extract Authorization header — look for "Bearer <token>"
3. If missing → return 401 immediately (do not forward)
4. Validate JWT signature using the secret/public key
5. If expired or invalid → return 401 with error message
6. If valid → extract claims, add as headers (X-User-Id, X-User-Role)
7. Forward the request with enriched headers to the upstream service

LIVE CODING · STEP 1

JWT dependency

pom.xml

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

LIVE CODING · STEP 2

JWT Utility — Validating Tokens

JwtUtil.java

```
@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(secret.getBytes(UTF_8));
    }

    public Claims validateToken(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token)
            .getBody();  // throws if invalid or expired
    }
}
```

LIVE CODING · STEP 3

JWT Authentication GlobalFilter

JwtAuthFilter.java — variables declaration

```
@Component
public class JwtAuthFilter implements GlobalFilter, Ordered {
    private final AntPathMatcher pathMatcher = new AntPathMatcher();
    private final JwtUtil jwtUtil;

    public JwtAuthFilter(JwtUtil jwtUtil) {
        this.jwtUtil = jwtUtil;
    }

    private record PublicRoute(HttpMethod method, String pathPattern) {
    }

    private static final List<PublicRoute> PUBLIC_ROUTES = List.of(
            new PublicRoute(HttpMethod.GET, "/api/v1/products/**"),
            new PublicRoute(HttpMethod.GET, "/actuator/health"),
            new PublicRoute(HttpMethod.POST, "/actuator/info")
    );









}
```

LIVE CODING · STEP 3

JWT Authentication GlobalFilter

JwtAuthFilter.java — filter() method

```
public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
    HttpMethod method = exchange.getRequest(). getMethod();
    String path = exchange.getRequest().getURI().getPath();
    if (isPublicRoute(method, path)) return chain.filter(exchange);  // Step 1
    String authHeader = exchange.getRequest().getHeaders().getFirst(HttpHeaders.AUTHORIZATION);
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        return unauthorizedResponse(exchange, "Missing token");  // Step 2-3
    }
    try {
        Claims claims = jwtUtil.validateToken(authHeader.substring(7));  // Step 4
        String userId = claims.getSubject();
        String role = claims.get("role", String.class);
        if (role == null || role.isBlank()) {
            return unauthorizedResponse(exchange, "Token does not contain role claim");
        }
        ServerHttpRequest enriched = exchange.getRequest().mutate()
                .headers(headers -> {
                    headers.remove("X-User-Id");
                    headers.remove("X-User-Role");
                    headers.add("X-User-Id", userId);
                    headers.add("X-User-Role", role);
                }).build();
return chain.filter(exchange.mutate().request(enriched).build());  // Step 7
    } catch (JwtException e) {
        return unauthorizedResponse(exchange, "Invalid or expired token");  // Step 5
    }
}
```

JwtAuthFilter.java — isPublicRoute(), unauthorizedResponse(), getOrder() methods

```
    private boolean isPublicRoute(HttpMethod method, String path) {
        return PUBLIC_ROUTES.stream().anyMatch(route ->
                route.method().equals(method)
                        && pathMatcher.match(route.pathPattern(), path)
        );
    }

    private Mono<Void> unauthorizedResponse(ServerWebExchange exchange, String message) {
        ServerHttpResponse response = exchange.getResponse();
        response.setStatusCode(HttpStatus.UNAUTHORIZED);
        response.getHeaders().add(HttpHeaders.CONTENT_TYPE, "application/json");
        String body = """
                {
                  "status": 401,
                  "error": "Unauthorized",
                  "message": "%s"
                }
                """.formatted(message);
        DataBuffer buffer = response.bufferFactory().wrap(body.getBytes(StandardCharsets.UTF_8));
        return response.writeWith(Mono.just(buffer));
    }

    @Override
    public int getOrder() {
        return Ordered.HIGHEST_PRECEDENCE + 1; // runs after LoggingFilter
    }
```

LIVE CODING · STEP 4

Add JWT secret to config

application.yml

```
jwt:
  secret: microservices-pro-course-secret-key-2024-minimum-256-bits

# NOTE: In production, this comes from HashiCorp Vault (Session 21)
# NEVER commit real secrets to Git — this is a dev placeholder only
```

CONFIGURATION PATTERN

Public Routes — Whitelist, Not Hardcode

Today's implementation (simple)

```
List.of("/api/products")
```

In-code list — fine for today's lab, but rigid

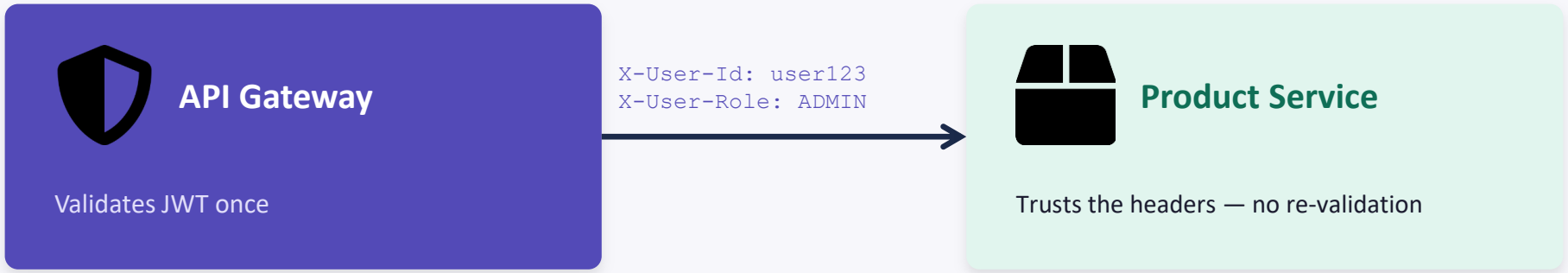
Production pattern (bonus)

```
@ConfigurationProperties
("gateway.public-routes")
```

Loaded from application.yml — change without recompiling

Bonus opportunity: implement the production pattern for +5 points in Lab 2B.

Header Enrichment — Gateway to Services



Why does the service trust these headers instead of re-validating the JWT?

The Gateway is the trust boundary. Everything inside the network trusts the Gateway's validation — re-validating in every service duplicates work and adds latency. This is THE core security principle of API Gateways.

DEMO

Testing JWT — 3 Scenarios

`GET /api/products (no token)` → **200 OK**

Public route — bypasses JWT validation

`POST /api/products (no token)` → **401 Unauthorized**

Protected route — token required

`POST /api/products (valid JWT)` → **201 Created**

Token validated, headers enriched, forwarded

LIVE CODING · STEP 4

Generating a Test JWT (for live demo)

```
# Option 1: Use jwt.io website
# Header: {"alg": "HS256", "typ": "JWT"}
# Payload: {"sub": "user123", "role": "CUSTOMER", "exp": <future timestamp>}
# Secret: microservices-pro-course-secret-key-2024-minimum-256-bits

# Option 2: Quick Java snippet for generating a test token
String token = Jwts.builder()
    .setSubject("user123")
    .claim("role", "CUSTOMER")
    .setExpiration(new Date(System.currentTimeMillis() + 3600000)) // 1 hour
    .signWith(Keys.hmacShaKeyFor(secret.getBytes()))
    .compact();

# Then call:
# curl -H 'Authorization: Bearer <token>' http://localhost:8080/api/products
```

Why Rate Limiting Belongs at the Gateway

Scenario: A bot sends 10,000 requests/second to GET /api/products

✕ Without rate limiting: Product Service is overwhelmed, database crashes, ALL users affected

✓ With rate limiting at Gateway: bot gets 429 after 100 req/s, Product Service stays healthy

Key insight

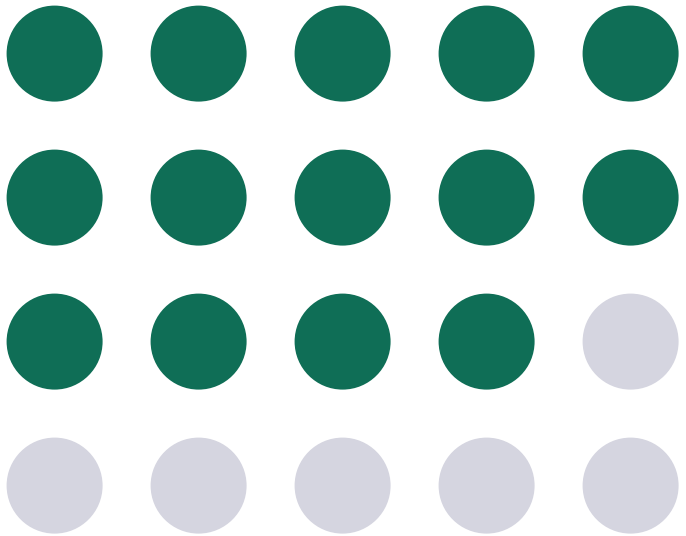
Rate limiting is a cross-cutting concern → belongs at the Gateway, not in services.

Services should NOT implement their own rate limiting — that defeats the purpose of the Gateway.

CORE CONCEPT

Token Bucket Algorithm

The Bucket



14 / 20 tokens remaining

`replenishRate = 10`

Add 10 tokens per second to the bucket

`burstCapacity = 20`

Bucket holds max 20 tokens

`Each request = 1 token`

If bucket is empty → 429 Too Many Requests

Steady: 10 req/s allowed · Burst: up to 20 at once · Redis stores token count per client

LIVE CODING · STEP 1

Redis Reactive dependency

pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
</dependency>
```

LIVE CODING · STEP 2

Configure Redis connection

application.yml

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
      # In production: host from Config Server, password from Vault (Session 21)
```

LIVE CODING · STEP 3

KeyResolver — Identifying Clients

RateLimitConfig.java

```
@Configuration
public class RateLimitConfig {

    @Bean @Primary  // Option A: rate limit per IP address
    public KeyResolver ipKeyResolver() {
        return exchange -> Mono.justOrEmpty(
            exchange.getRequest().getRemoteAddress()
        ).map(addr -> addr.getAddress().getHostAddress());
    }

    @Bean  // Option B: rate limit per authenticated user
    public KeyResolver userKeyResolver() {
        return exchange -> Mono.justOrEmpty(
            exchange.getRequest().getHeaders().getFirst("X-User-Id")
        ).defaultIfEmpty("anonymous");
    }
}
```

LIVE CODING · STEP 4

Adding RequestRateLimiter to the Route

application.yml — product-service route

```
spring:
  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://PRODUCT-SERVICE
          predicates:
            - Path=/api/products/**
          filters:
            - AddResponseHeader=X-Platform, microservices-pro
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
                redis-rate-limiter.requestedTokens: 1
                key-resolver: "#{@ipKeyResolver}"
```

DEMO

Testing Rate Limiting — 25 Rapid Requests

Shell loop — 25 requests as fast as possible

```
for i in {1..25}; do curl -o /dev/null -w "%{http_code}\n" http://localhost:8080/api/products; done
```

Expected output:

```
200 200 200 200 200 200 200 200 200 200
200 200 200 200 200 200 200 200 200 200
429 429 429 429 429
```

Notice X-RateLimit-Remaining decreasing to 0. The Gateway protects Product Service — it never even sees the rejected requests.

CLOSING THE LOOP FROM SESSION 2

StripPrefix — Why We Set It to 0

Scenario: needs stripping

```
Client calls:
  GET /gateway/products/123

StripPrefix=1
  removes 1 segment: /gateway

Service receives:
  GET /products/123  □
```

Our platform: no stripping needed

```
Client calls:
  GET /api/products/123

StripPrefix=0
  removes nothing

Service receives:
  GET /api/products/123  □
```

HANDS-ON

Lab 2B — JWT Authentication Filter & Rate Limiting

1	**Implement JWT Authentication Filter**	JwtUtil + JwtAuthFilter, public route bypass, header enrichment
2	**Add Rate Limiting to Product Route**	RequestRateLimiter with IP-based KeyResolver
3	**Write Unit Tests**	Minimum 5 tests covering auth + rate limit behaviour

PLATFORM PROGRESS

What We Added to the Platform Today

Capability Added

Security & Abuse Protection Layer

Gateway now validates JWT tokens and limits request rates — services behind it are protected.

Added to api-gateway

- JwtUtil — token validation with JJWT
- JwtAuthFilter — GlobalFilter, order 1
- RateLimitConfig — IP + User KeyResolvers
- RequestRateLimiter on product-service route

- Session 1: Config + Eureka + Product Service
- Session 2: API Gateway — Single Entry Point
- Session 3: Gateway secured — JWT + Rate Limiting

Config	Eureka	Product	Gateway Secured	Payment	Inventory	Order	Notif.

Daily Quiz

8 Questions · 10 Minutes

Topics: JWT Validation · Rate Limiting · Token Bucket · Filter Order · Security Boundary

BEFORE SESSION 4

Checkpoint Commit & Homework

Checkpoint — Definition of Done

- ☐ GET /api/products (no token) → 200
- ☐ POST /api/products (no token) → 401
- ☐ Valid JWT → 201 Created
- ☐ 25 rapid requests → some return 429
- ☐ Pushed: session-03: ...

Pre-Session 4 Reading (15 min)

Resilience4j — Circuit Breaker Basics
resilience4j.readme.io

Knowledge Check Questions for Session 4:

- What is a cascading failure?
- What are the 3 Circuit Breaker states?
- What is a fallback method?

TROUBLESHOOTING

Common Issues & Solutions

"No KeyResolver bean" error	Add @Primary to your main KeyResolver bean — Spring needs to choose one
Rate limiter not triggering (always 200)	Check Redis is running: docker compose ps. Verify YAML indentation under redis-rate-limiter.*
JWT 401 on a valid token	Check jwt.secret matches the signing secret. Minimum 256 bits (32 characters)
"JWT strings must contain exactly 2 periods"	Token is malformed — ensure Bearer prefix is stripped: token = authHeader.substring(7)
Public route returning 401	Use path.startsWith(), not path.equals() — path may include trailing segments

Session 3 Complete

The Gateway is now a Security & Protection layer

Next: Session 4 — Resilience Patterns: Circuit Breaker & Retry · Monday, 3:00–5:30 PM, Online